

Bar-Ilan University

אוניברסיטת בר-אילן

M.Sc. Research Proposal

הצעת מחקר לתואר שני

Rule Learning with Small Language Models
in Interactive Puzzle Games

למידת חוקי משחק באמצעות מודלי שפה קטנים
במשחקי חידה אינטראקטיביים

Student: סטודנט
Adam Celermajer
אדם סלרמאג'ר

Supervisors: מנחים
Prof. Gal Kaminka
Prof. Yoav Goldberg

Department of Computer Science and Artificial Intelligence
המחלקה למדעי המחשב ובינה מלאכותית
Bar-Ilan University, Ramat Gan, Israel

October 2026

Contents

1 Introduction **1**

2 Related Work **1**

3 Research Goal and Hypotheses **3**

3.1 The task . . . 3

3.2 Example: Modality . . . 3

3.3 Capabilities . . . 3

3.4 Objective . . . 4

4 Initial Results **5**

4.1 Pilot: the Qwen 3.8 family on Modality (single runs) . . . 5

4.2 The 27B direct-play gap across games . . . 6

4.3 Whole-game solving and mechanics recovery by larger models . . . 6

4.4 Goal-recognition data . . . 7

4.5 The platform that produced these results . . . 7

5 Work Plan **8**

5.1 First: compare the Qwen 3.8 family on one fixed harness . . . 8

5.2 Second: locate the bottleneck through controlled interventions . . . 8

5.3 Third: build and ablate the support that makes the small model solve . . . 9

5.4 Validation, efficiency and generalization . . . 9

5.5 Timeline, deliverables and resources . . . 9

References **11**

1 Introduction

This thesis will enable a small language model to learn game rules from images, grid observations and interaction, and use those rules to solve interactive puzzles. Our primary target is Qwen 3.8-27B. The deliverable is a working agent that discovers the controls, goal and mechanics of a new game, constructs a useful world model, and produces solutions that succeed when executed independently. We will use controlled experiments to identify the capabilities the harness must support and demonstrate the resulting improvement on games held out from development.

The task is more demanding than solving a puzzle with stated rules. In our main game-playing conditions, the agent has access to both rendered PNG images and integer grid observations, together with anonymous action identifiers. Images provide visual evidence about objects, terrain and targets; they do not state the transition rules, action meanings or win condition. The agent must combine visual and symbolic evidence, design informative experiments, infer what counts as success, and plan with incomplete knowledge. Later levels test whether the learned rules survive changes in layout and combinations of mechanics. This is the skill-acquisition setting motivating ARC-AGI-3 (ARC Prize Foundation, 2026; Chollet, 2019).

The research foundation exists: an ARC-compatible PuzzleScript environment, diagnostic evaluation artifacts, persistent direct-play baselines, an isolated adaptation of Agentic Variation Operators (AVO) (Chen et al., 2026a), and a workbench that traces a solver's result back to the observations, experiments and revisions that produced it (Section 4).

The existing results establish both feasibility and a concrete research target. AVO-generated solvers have completed all levels of Modality, Knightoban and IceCrates in fresh environments, including solvers produced by DeepSeek V4.1 Flash, and a single Qwen 3.8 Max campaign completed Modality. In contrast, the completed Qwen 3.8-27B direct-play pilot achieved 0/4, 0/6 and 3/6 levels on those games, and under AVO it reached 2/4 on Modality with a solver that misread its own observations. Its recorded behavior reveals specific failures in action interpretation, goal recognition and hypothesis revision. Every comparative number so far is a single run. We will close this gap by turning those diagnoses into tested components of a small-model learning system, with repeated runs on one fixed harness.

2 Related Work

Interactive skill acquisition. ARC-AGI-3 evaluates learning through interaction with unfamiliar environments, including exploration, goal inference and planning (ARC Prize Foundation, 2026). PuzzleJAX establishes PuzzleScript as a source of diverse reasoning and learning tasks (Earle et al., 2025). Our platform retains the original engine as the transition authority and exposes an ARC-compatible interface. This lets us study how an agent acquires mechanics while separately measuring predictive knowledge and executable solutions. The experiments proposed here concern PuzzleScript tasks; they do not claim a score on the official ARC-AGI-3 benchmark.

Learning game dynamics from interaction. Theory-based reinforcement learning shows that an agent which induces an explicit theory of objects, rules and goals and plans with it reaches human-like sample efficiency on novel video games (Tsividis et al., 2021). Program-synthesis world models extend this to language models: WorldCoder (Tang et al., 2024) writes and repairs a Python transition model from interaction, code world models guided by Monte Carlo tree search (Dainese et al., 2024) select among candidate programs by their predictions, and PoE-World (Piriyakulkij et al., 2025) composes many small programmatic experts into a model of Atari-style games from a few demonstrations. All of these use frontier models as the program writer. We share the executable-model representation but target a fixed small model

and ask which capability prevents it from learning.

Hypothesis formation and revision in language models. Language models propose and refine hypotheses well when the loop is externally structured: Hypothesis Search generates natural-language rules for ARC tasks, turns them into programs and keeps those that pass the examples (Wang et al., 2024). They do poorly when asked to find a rule and then apply it themselves: iterative refinement improves the proposals, but the models remain weak at executing their own rules (Qiu et al., 2024). Our 27B traces show the second pattern: a wrong explanation of an accidental win is retained across levels. The proposed experiment-and-revision components target exactly this.

Agent interfaces and persistent learning. ReAct interleaves reasoning and actions to support grounded decision making (Yao et al., 2023). SWE-agent demonstrates that the interface provided to a language model affects its ability to carry out a task (Yang et al., 2024). Reflexion and Voyager add verbal self-revision and a persistent skill library across episodes (Shinn et al., 2023; Wang et al., 2023). We investigate this principle under a different constraint: the model must learn the task's mechanics and objective rather than follow a stated specification. Our direct-play and coding-agent runners provide an existing basis for controlled comparisons of memory, tools, observation formats and experiment structure.

Executable models and evolutionary improvement. FunSearch evolves programs scored by an external evaluator (Romera-Paredes et al., 2024). AVO replaces the sampled mutation with autonomous coding agents as variation operators that propose and improve implementations with execution feedback (Chen et al., 2026a); the same architecture has been reported to solve ARC-AGI-3 with frontier models (Chen et al., 2026b), and RSIAgent studies autonomous exploration for self-improvement in new environments (Zhu et al., 2026). Our adaptation combines game research, strategy evolution and a separate fresh-solver evaluation. This creates a testable connection between a model's hypotheses and an executable artifact: a candidate earns its score only when its solver succeeds independently.

Benchmarks for agents in games. BALROG (Paglieri et al., 2025) and SmartPlay (Wu et al., 2024) evaluate language agents on long-horizon games with stated rules and report that vision-based variants lag their text counterparts; Baba Is AI (Cloos et al., 2024) requires agents to manipulate the rules themselves and finds failures when rules must be combined; WorldTest (Warrier et al., 2026) separates an interaction phase from queries about the learned world model. PuzzleJAX (Earle et al., 2025) and ARC-AGI-3 supply unknown rules. Unlike these, our platform scores a learned rule against the game source by replaying counterfactual actions in the real engine, separating rule accuracy from task success.

Harness effects and format sensitivity. Interface choices change what a model can do: enforced output formats degrade reasoning (Tam et al., 2024), prompt formatting changes scores by many points (Sclar et al., 2024), and visual harnesses alter agent behaviour (Han et al., 2026). Agent evaluations must also control cost (Kapoor et al., 2024). Our JSON-schema diagnostic is an instance of the first effect; our resource reporting follows the last.

Scale and measurement. Scaling laws describe loss, not task competence (Kaplan et al., 2020), and abilities that appear to emerge abruptly with scale (Wei et al., 2022) can be artefacts of discontinuous metrics (Schaeffer et al., 2023). Whether a 27B model "can" learn a game therefore depends on how learning is measured. We use graded measures, levels completed and

counterfactual rule accuracy, rather than a binary solved flag, and we do not treat a difference between Qwen 3.8 sizes as a scale effect without the controls in Section 5.

Prior knowledge, small open models and training exposure. Human game learning depends partly on object and game conventions (Dubey et al., 2018). Our traces suggest that recognizing conventions such as undo and box pushing can simplify discovery substantially. However, whether a model has seen similar environments cannot be read off its behaviour; contamination studies (Golchin and Surdeanu, 2024; Elazar et al., 2024) show how hard attribution is even with open corpora. We therefore treat pretraining as a bounded, not identified, factor (Section 3), and test whether explicit exploration and verification allow the 27B model to acquire the missing knowledge during play.

Position. The contribution is not another world-model agent. It is (i) a capability-level diagnosis of *why* a fixed small open model fails, with rule accuracy audited against the game source by counterfactual replay; (ii) controlled interventions that supply ground truth as diagnostics; and (iii) an evaluation that separates rule accuracy from winning.

3 Research Goal and Hypotheses

3.1 The task

PuzzleScript is a language for turn-based grid games (Lavelle, 2013). A game defines objects and their appearance, rewrite rules for state transitions, a win condition, and an ordered sequence of levels. The resulting tasks include box-pushing puzzles, sliding motion, objects with different movement rules, mode changes, and interactions whose effects depend on terrain or history. Their compact appearance makes the mechanics inspectable by an evaluator while leaving substantial discovery and planning work for an agent.

Our collection contains 206 game entries; experiments use curated, versioned playable subsets. The agent sees neither the game identity nor the source. The encoding `visible-cell-v1` assigns an anonymous integer to each rendered tile texture; the direct player receives the rendered PNG alongside the grid at each turn, and AVO exposes rendered observations through its research interface. Object-name legends and action meanings are withheld. Identical visible textures may conceal different underlying states, so the agent may need observation history rather than a transition model based on the latest frame alone.

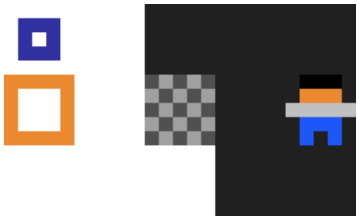
3.2 Example: Modality

Figure 1 illustrates the distinction between seeing a board and understanding a game. In Modality, the player cannot move directly between white and black terrain; a checkered tile connects them. Crate pushing depends on compatible terrain, the goal is to place crates on targets, and `ACTION7` undoes a move. None of this is stated: calling undo a jump changes the interpretation of earlier transitions, and calling a crate the goal sends plans toward the wrong objective.

3.3 Capabilities

We decompose the task into the seven capabilities in Table 1. The decomposition connects the experimental measurements to concrete design decisions. It also distinguishes a failure to learn a game from a failure of the software protocol.

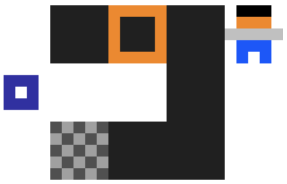
These capabilities interact. A wrong goal can invalidate a correct movement model; an incorrect interpretation of undo can make a recoverable planning error appear permanent. The



(a) First-level image

```
0  1  2  2  2
3  1  4  2  5
1  1  1  2  2
```

(b) Anonymous tile grid



(c) A later layout

Figure 1: Modality observations. For the evaluator, the initial IDs denote a target (0), white floor (1), black floor (2), crate (3), bridge (4) and player on black floor (5). These meanings are not supplied to the agent. Later levels require reusing the inferred mechanics on new configurations.

Capability	Required behavior	Diagnostic evidence
Perception	Ground objects in images and grids; track changing textures and hidden state.	Object tracking and next-frame predictions.
Action interpretation	Discover movements, undo, selection and commitment.	Controlled action tests and the agent's stated control model.
Rule induction	Learn conditional effects, including interactions.	Predicted versus observed transitions; tests distinguishing rival rules.
Goal recognition	Infer the actual win condition.	Goal statements and hidden, elicited and supplied-goal conditions.
Planning	Find legal solutions and recover from dead ends.	Fresh solver evaluations, search traces and recovery actions.
Transfer	Reuse and refine knowledge across levels.	Performance when familiar mechanics appear in new layouts.
Credit assignment	Explain why an action sequence produced a win.	Post-win reconstruction checked against the winning transition.

Table 1: Capabilities supporting rule learning and its use in planning.

proposed agent will maintain explicit links between observations, hypotheses and plans so that contradictory evidence causes a revision rather than a new local explanation for every failure.

3.4 Objective

We will build a small-model rule-learning agent that infers game dynamics from image and grid transitions and produces working puzzle solutions. Qwen 3.8-27B is the primary development model. We will keep its weights fixed while developing the harness, so improvements can be attributed to the agent system. Larger models provide diagnostic comparisons and successful learning traces; the final small-model condition will not rely on a larger model selecting its actions or revising its solver at evaluation time. The thesis therefore manipulates one factor, the harness, with the model fixed; model size and pretraining exposure are bounded by controls, not identified.

The harness will support a complete learning cycle: formulate a hypothesis, choose a discriminating experiment, compare a predicted transition with the real outcome, revise the world model, and plan using the verified knowledge. Memory will retain the evidence for a rule and its unresolved cases, not only a conclusion. In the coding condition, learned knowledge will be converted into a solver that is tested from a fresh reset.

Three research questions guide construction:

1. Which capabilities distinguish correct rule acquisition and revision from the observed failures of the small model?
2. Which of these capabilities can be supplied or strengthened through exploration, memory, verification and planning support?
3. Do the learned rules predict held-out transitions and support reliable puzzle solutions, including on games held out from harness development?

Testable hypotheses.

H1: Learning structure. An explicit experiment-and-revision loop improves rule prediction and whole-game success over the direct-play baseline with the same small model and provider.

H2: Goal and control bottlenecks. Correcting goal recognition or action interpretation removes a substantial part of the observed failures. Separate interventions will measure which component matters in each game.

H3: Externalized planning. Executable world models, deterministic search and fresh evaluation improve solution reliability beyond verbal planning alone.

H4: Capability gaps are localisable. Under one fixed harness, the Qwen 3.8 models fail at identifiable stages (observation parsing, action interpretation, rule consolidation, verification) rather than uniformly, and the stage at which 27B fails is the one whose support most improves it. We do not claim to separate parameter count from architecture, post-training or serving; a same-size dense model from another family and a zero-shot next-frame probe bound, but do not identify, those factors.

4 Initial Results

We report results first and the platform that produced them last. Every comparative number below comes from a single run per model and condition.

4.1 Pilot: the Qwen 3.8 family on Modality (single runs)

Table 2 reports one run per model and condition on Modality, all with high reasoning effort and first-party Alibaba routing. Baseline+ is direct play with note tools and a 100-action budget; AVO allows 300 research actions, two real solver self-checks and a 1,500-action final evaluation of a frozen solver. The conditions are not resource-matched and each cell is a single run; the table motivates the comparisons in the Work Plan rather than establishing a ranking. Every recorded action (3,700 across eight logs) was replayed in the native PuzzleScript engine with no position or level mismatch.

Model	Baseline+ (100 actions)	AVO research	AVO final solver	Solver actions
Qwen 3.8 Max	1/4	3/4 (123 actions)	**4/4**	265
Qwen 3.8 Flash	1/4	3/4 (160 actions)	1/4	1,500
Qwen 3.8-27B	0/4	1/4 (234 actions)	2/4	1,500

Table 2: Modality, October 2026, $n = 1$ per cell. The three baseline+ runs share harness, provider, effort, budget and observation format (the 27B run is five days older). Flash is a 125B mixture-of-experts model with 6B active parameters; 27B is dense. Level 4 was visible to Max during research, so the 4/4 is not a held-out result. The harness used has since been retired (Section 5).

Three verified observations shape the proposal. First, winning did not require a correct rule: Max's solver models the checkered tile as a global phase toggle and exempts target cells from the terrain test, and counterfactual replay in the real engine finds 32 wrong predictions

among 1,060 alternative moves at its own decision states (27 too strict, 5 too permissive); none of those situations arose on its 265-action path. Second, Flash's research planner encoded the exact player-movement rule (no direct move between white and black floor, though not the push rule) and used it to clear levels 2–3, but never wrote it into its notes or world model; its final solver relearned the same movement rule online within 80 actions and still spent 1,490 actions oscillating on level 2, so the cause of its 1/4 is open. Third, 27B's solver parsed the older of the two frames in each observation, built delayed-action mechanics that fit its own misreading, and chose a position no-op in 1,405 of its 1,420 level-3 actions; in direct play it read undo as a "jump" and declared a recoverable deadlock permanent. These are recorded behaviours, not demonstrated causes; isolating causes is the first stage of the Work Plan.

4.2 The 27B direct-play gap across games

The Qwen 3.8-27B direct-play pilot (Table 3) used the same condition on three games: baseline+ through the Codex harness routed to OpenRouter, Alibaba provider, high effort, image and grid observations. Each run reached its action budget, so the partial IceCrates result is directly distinguishable from a startup or transport error.

Game	Budget	Levels	Harness / provider	Evidence for the next intervention
Modality	100	0/4	Codex / Alibaba	Undo misread; crate deadlock not recovered
Knightoban	120	0/6	Codex / Alibaba	Goal and special movement not identified
IceCrates	120	3/6	Codex / Alibaba	Undo identified; later mechanics unresolved

Table 3: Completed 27B direct-play pilot, October 2026, one run per game ($n = 1$), all on the Codex harness via OpenRouter with the Alibaba provider. AVO and direct-play results are distinct conditions; their difference motivates, rather than replaces, matched comparisons.

In IceCrates the same model identified undo and solved three levels in its first 27 actions, so the research target is the reliable acquisition and revision of specific mechanics, not an assumption that the model has no game-playing ability. We have also isolated one interface effect: in a small controlled diagnostic, an enforced JSON action schema yielded zero note-tool calls in two checks, whereas the corresponding checks without that schema each called the tool. The action parser now accepts valid actions surrounded by prose and requests a correction for unusable replies; provider pinning and recorded request metadata reduce routing ambiguity. These checks are reported separately from game-learning outcomes.

4.3 Whole-game solving and mechanics recovery by larger models

Table 4 reports representative best fresh evaluations. These are independently executed solvers, not scores inferred from research notes or from the model's claim that it has understood a game.

Game	Player / optimizer	Levels	Solver actions
Modality	GPT-5.6 Luna / GPT-6 Astra	4/4	263
Knightoban	GPT-5.6 Luna / GPT-6 Astra	6/6	255
IceCrates	GPT-6 Astra / GPT-6 Astra	6/6	138
Modality	DeepSeek V4.1 Flash / same	4/4	263
Knightoban	DeepSeek V4.1 Flash / same	6/6	216
IceCrates	DeepSeek V4.1 Flash / same	6/6	90

Table 4: Observed AVO whole-game successes. Rows report the best completed fresh evaluation from the corresponding campaign, not averages across repeated attempts ($n = 1$ per row). Model-role settings are retained in the run manifests.

We audited the discovered mechanics against the source rules after the runs. In Modality, controlled tests established terrain-dependent movement and pushing. In Knightoban, the agent separated move preview, spatial selection and action-key commitment and constructed a planner for interacting pieces. In IceCrates, experiments distinguished moving crates from walls, established water filling and the need to stop on the goal, and corrected a missing rule for pushing a two-crate train; after that correction, the independent solver completed all six levels. The audit relies on contemporaneous strategies, transitions and solver code and establishes the mechanics needed by the successful solvers, not that every unobserved combination was characterized. GPT-5.6 Luna also solved Feeling Like Filling (6/6), Mowing (4/4) and Fatigued Square Maze (8/8) in both recorded repeats of the six-game comparison, and Enqueue reached 8/14 levels with both the GPT-6 Luna and DeepSeek Flash configurations. These supply successes and partial-progress cases for testing whether small-model improvements transfer across mechanics.

4.4 Goal-recognition data

We also completed data collection for a separate goal-recognition study. Four models (Claude Opus 4.8, GPT-5.5, Kimi K2.6 and DeepSeek V4 Pro) described the goal and win condition of 50 curated games from an initial frame or an initial frame followed by three random actions, in text or text-plus-image conditions (700 predictions); 38 human participants gave 114 goal descriptions across 48 games. Correctness annotation has not started, so no accuracy is claimed. The collection exposed a representation confound: initial-frame text prompts used game-specific glyphs while trajectory prompts used anonymous IDs, and a model read `@` as a player when it denoted a crate on a target. The Work Plan bounds the analysis to one condition with this difference explicit.

4.5 The platform that produced these results

The implementation separates the game engine from the agent interface. A Node.js runtime executes the PuzzleScript rules; a Python service adapts it to the ARC interaction contract, including resets, actions and scorecards, and the same agent code can address an official ARC backend. Evaluator artifacts measure knowledge independently of winning: the benchmark set covers 21 games with replay-verified reference trajectories and ten unique next-observation probes per game, scored by exact-frame accuracy, cell accuracy and transition-edit measures; no model has been scored on it yet. The workbench records settings, process ownership, frames, transitions, note revisions, model requests and solver files for every run, and replays any action with the reasoning that produced it.

An earlier explicit rule-induction agent records state–action–state transitions, asks a model to propose rules, compiles them into an executable logical form and checks them against the evidence; its saved Sokoban experiments hold verified movement, collision and crate-push rules, and it is the baseline representation for the proposed learning cycle.

Direct play. The direct player maintains one model session across the game. The baseline+ condition adds persistent note tools and a strategy file while retaining the same action and observation loop, with recorded transitions and immutable note revisions.

AVO adaptation. We implemented a distinct research–optimization–evaluation architecture. An inner coding agent explores the game, compares continuations from saved checkpoints and constructs a solver; an outer optimizer improves strategies through a recorded candidate lineage; a trusted evaluator then executes each frozen solver from a fresh reset, without language-model

calls. The candidate's fitness is the number of levels completed in this independent evaluation:

$$\text{strategy}_i \longrightarrow \text{research}_i \longrightarrow \texttt{solver}_i \longrightarrow \text{fresh evaluation.}$$

Model commands run in a sandbox without access to game sources, evaluation artifacts or the network; the trusted controller owns environment and provider communication and retains strategies, feedback and world-model reports, whose presence does not affect fitness.

5 Work Plan

5.1 First: compare the Qwen 3.8 family on one fixed harness

The October pilot (Table 2) used a Codex-based harness routed through OpenRouter that has since been retired. The confirmatory comparison will fix one harness (the OpenCode or bare runner, chosen and frozen before the first run), Alibaba routing, high effort, image and grid observations, a 100-action direct-play budget and the AVO budgets above, with three independent runs per model and game. Harness version is recorded as an experimental factor and never changed mid-comparison. The same model occupies the player and optimizer roles in AVO, and direct-play and AVO outcomes remain separate, with their action, candidate, time and token budgets reported explicitly.

This experiment identifies the next useful intervention. If Max discovers the decisive rule but 27B does not, we will compare the experiments and revisions that led to it. If the same smaller model succeeds under AVO, we will isolate which support made the difference. If the family stalls on the same mechanic, we will target that shared bottleneck. Family differences do not isolate parameter count: two controls bound size and pretraining without identifying them, a dense open model of similar size from another family on the same harness, and the zero-shot next-frame probe on the existing 21-game benchmark. The comparison covers Modality, Knightoban and IceCrates, which supply distinct challenges in terrain, selection and commitment, and multi-object sliding interactions.

5.2 Second: locate the bottleneck through controlled interventions

We will use the capability decomposition to select interventions, rather than add several forms of assistance at once. Initial conditions will test:

1. **Goal recognition:** the goal is hidden, explicitly elicited and revised, or supplied by the evaluator.
2. **Action interpretation:** controls remain anonymous, or their true meanings are supplied while the rules and goal remain hidden.
3. **Mechanics and planning:** selected true mechanics are supplied diagnostically, or the model's learned mechanics are paired with a deterministic planner.

Supplying the truth is a diagnostic intervention, not the final task setting. A gain under a supplied-goal condition estimates how much incorrect goals contribute to failure; a residual gap motivates further tests of dynamics and planning. The final agent must acquire the hidden information itself.

For each failed attempt we will identify the first consequential mismatch between the model's belief and the observed transition, along with secondary causes such as failure to recover. Annotators will cite the supporting step, action and note rather than infer the cause from terminal reward alone. Independent human annotation of a sample will assess the reliability of the taxonomy; we will report agreement and disagreements rather than treat an automatic label as ground truth.

The goal-recognition corpus is a bounded side task: annotate the initial-frame condition (50 games, four models) against the game win conditions with a correct/partial/incorrect rubric and report model–human agreement with the representation difference explicit.

5.3 Third: build and ablate the support that makes the small model solve

We will develop the harness from the diagnosed failures. Candidate components are a ledger linking hypotheses to evidence, controlled experiment selection, explicit transition predictions, revision when evidence contradicts a rule, and reconstruction of what caused each win. The ledger will distinguish observed, predicted and untested cases, preserving uncertainty instead of recording unknown moves as walls or no-ops.

The coding condition will support executable world models and search. The small model will infer and implement the mechanics; deterministic procedures will handle repeated simulation and path search. A mismatch between simulation and a real transition will invalidate the relevant rule or plan and trigger new research. Fresh solver evaluation will verify that the resulting program works without continued language-model guidance.

Each component will first be compared with the established baseline under matched conditions. We will then combine components that produce repeatable gains and remove them individually to determine their contribution. The comparison will record the actual observation history, context policy and tool inventory, since changing these silently can change the task.

5.4 Validation, efficiency and generalization

Let $L(m, h, g)$ be the number of levels completed by model m with harness h on game g, and let n_g be the number of playable levels. Two outcomes are primary: L/n_g on held-out layouts, and counterfactual rule accuracy $A(m, h, g)$: for each recorded decision state the real engine is restored and all legal actions are executed; A is the fraction the learned model predicts correctly. Every table reports both, so that a win obtained with a wrong model (as in the Max pilot) is visible. We will also report whole-game success, actions, candidate evaluations, elapsed search time, tokens and measured API cost. For AVO, the solver outcome comes from the fresh frozen solver; research progress is an additional diagnostic. Next-observation accuracy and goal correctness measure learned knowledge separately from task completion.

Development uses the three diagnostic games. Before final evaluation, we will select and freeze a held-out subset from the curated collection, covering different mechanics and level structures. Game sources, verified solutions and source-derived annotations remain evaluator-only. Runs are independent, and no notes or solutions from a held-out game are inherited by its evaluated agent. We will report repeated-attempt outcomes as counts (k/n solved, median levels) and uncertainty, not only the best candidate.

The primary success criterion is that the fixed small model learns rule models that predict held-out transitions and enable complete solutions on games its baseline fails, with repeatable gains under declared resource budgets. A solver that improves only on a development layout is an intermediate result; the final contribution must demonstrate that the learning procedure transfers.

5.5 Timeline, deliverables and resources

Table 5 defines the remaining work by stage, with months, run counts and estimated cost. Family-comparison results will guide the interventions, with repeated experiments concentrated on promising conditions.

Stage	Work and completion evidence	Months	Runs	Est. cost
Family comparison	Freeze one harness; 3 models × 3 games × {baseline+, AVO} × 3 repeats, plus the two bounding controls. Evidence: matched settings, outcomes, rule accuracy, costs and decisive-rule trace comparisons.	Nov–Dec 2026	54	≈$700
Bottleneck analysis	Interventions on 27B and Flash: supplied goal, supplied actions, latest-frame-only observation, enforced self-check, rival-hypothesis step; 3 repeats plus controls. Evidence: scored interventions, annotated failure traces with agreement, bounded goal-corpus analysis.	Jan–Mar 2027	≈90	≈$400
Small-model solver	Implement and combine experiment, memory, verification and search support; ablations on the development games. Evidence: whole-game successes by the fixed small model; component ablations.	Apr–Jun 2027	≈60	≈$300
Generalization	Freeze the harness and held-out games; 6 games × 3 repeats × {baseline, harness}. Evidence: fresh solver scores, rule accuracy, uncertainty and efficiency comparisons.	Jul–Aug 2027	36	≈$200
Thesis and release	Document the method, artifacts and limits. Evidence: reproducible implementation, experiment records and thesis.	Sep–Oct 2027	—	—

Table 5: Timeline for the remaining research. Run counts are planned campaigns or direct-play runs. Costs are list-price estimates with no billing data behind them; Max dominates the first stage.

Resource management. We will enforce action, candidate and wall-time budgets, record tokens and measured costs, and retain unsuccessful attempts. Infrastructure errors will be distinguished from capability outcomes. All requests are recorded with model id, provider, effort and usage; the 27B model is open-weights and can be served locally for the final evaluation, while Max is a diagnostic comparator only.

Expected contributions. We will deliver a working rule-learning agent built around a small model, validated on held-out games by rule accuracy and whole-game success. The research will also provide a validated capability taxonomy, scored goal-recognition data, component ablations, and reproducible rule-prediction and fresh-solver evaluations.

References

ARC Prize Foundation. ARC-AGI-3: A new challenge for frontier agentic intelligence. *arXiv preprint arXiv:2603.24621*, 2026. URL https://arxiv.org/abs/2603.24621.

Terry Chen, Zhifan Ye, Bing Xu, Zihao Ye, Timmy Liu, Ali Hassani, Tianqi Chen, Andrew Kerr, Haicheng Wu, Yang Xu, Yu-Jung Chen, Hanfeng Chen, Aditya Kane, Ronny Krashinsky, Ming-Yu Liu, Vinod Grover, Luis Ceze, Roger Bringmann, John Tran, Wei Liu, Fung Xie, Michael Lightstone, and Humphrey Shi. AVO: Agentic variation operators for autonomous evolutionary search. *arXiv preprint arXiv:2603.24517*, 2026a. URL https://arxiv.org/abs/2603.24517.

Terry Chen, Yeyin Zhu, Zhifan Ye, Jean-Francois Puget, and Humphrey Shi. NVIDIA AVO reaches 100% on ARC-AGI-3, demonstrating a frontier-level general-purpose architecture for long-horizon autonomous agents. NVIDIA Technical Blog, 21 August 2026, https://developer.nvidia.com/blog/nvidia-avo-reaches-100-on-arc-agi-3-demonstrating-a-frontier-level-general-purpose-architecture-for-long-horizon-autonomous-agents/, 2026b.

François Chollet. On the measure of intelligence. *arXiv preprint arXiv:1911.01547*, 2019. URL https://arxiv.org/abs/1911.01547.

Nathan Cloos, Meagan Jens, Michelangelo Naim, Yen-Ling Kuo, Ignacio Cases, Andrei Barbu, and Christopher J. Cueva. Baba is AI: Break the rules to beat the benchmark. *arXiv preprint arXiv:2407.13729*, 2024.

Nicola Dainese, Matteo Merler, Minttu Alakuijala, and Pekka Marttinen. Generating code world models with large language models guided by Monte Carlo tree search. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2405.15383.

Rachit Dubey, Pulkit Agrawal, Deepak Pathak, Thomas L. Griffiths, and Alexei A. Efros. Investigating human priors for playing video games. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018. arXiv:1802.10217.

Sam Earle, Graham Todd, Yuchen Li, Ahmed Khalifa, Muhammad Umair Nasir, Zehua Jiang, Andrzej Banburski-Fahey, and Julian Togelius. PuzzleJAX: A benchmark for reasoning and learning. *arXiv preprint arXiv:2508.16821*, 2025. URL https://arxiv.org/abs/2508.16821.

Yanai Elazar, Akshita Bhagia, Ian Magnusson, Abhilasha Ravichander, Dustin Schwenk, Alane Suhr, Pete Walsh, Dirk Groeneveld, Luca Soldaini, Sameer Singh, Hannaneh Hajishirzi, Noah A. Smith, and Jesse Dodge. What's in my big data? In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.20707.

Shahriar Golchin and Mihai Surdeanu. Time travel in LLMs: Tracing data contamination in large language models. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2308.08493.

Qiushi Han, Keya Hu, Linlu Qiu, Cathy Wu, and Kaiming He. VISTA: A visual harness for reasoning in an interactive world. Project page, https://vista-research.github.io/, 2026.

Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.

Sayash Kapoor, Benedikt Stroebl, Zachary S. Siegel, Nitya Nadgir, and Arvind Narayanan. AI agents that matter. *arXiv preprint arXiv:2407.01502*, 2024.

Stephen Lavelle. PuzzleScript: An open-source HTML5 puzzle game engine. https://www.puzzlescript.net/, 2013. Software and domain-specific language.

Davide Paglieri, Bartłomiej Cupiał, Samuel Coward, Ulyana Piterbarg, Maciej Wołczyk, Akbir Khan, Eduardo Pignatelli, Łukasz Kuciński, Lerrel Pinto, Rob Fergus, Jakob Nicolaus Foerster, Jack Parker-Holder, and Tim Rocktäschel. BALROG: Benchmarking agentic LLM and VLM reasoning on games. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2411.13543.

Wasu Top Piriyakulkij, Yichao Liang, Hao Tang, Adrian Weller, Marta Kryven, and Kevin Ellis. PoE-World: Compositional world modeling with products of programmatic experts. *arXiv preprint arXiv:2505.10819*, 2025.

Linlu Qiu, Liwei Jiang, Ximing Lu, Melanie Sclar, Valentina Pyatkin, Chandra Bhagavatula, Bailin Wang, Yoon Kim, Yejin Choi, Nouha Dziri, and Xiang Ren. Phenomenal yet puzzling: Testing inductive reasoning capabilities of language models with hypothesis refinement. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.08559.

Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. Mathematical discoveries from program search with large language models. *Nature*, 625:468–475, 2024. doi: 10.1038/s41586-023-06924-6.

Rylan Schaeffer, Brando Miranda, and Sanmi Koyejo. Are emergent abilities of large language models a mirage? In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2304.15004.

Melanie Sclar, Yejin Choi, Yulia Tsvetkov, and Alane Suhr. Quantifying language models' sensitivity to spurious features in prompt design or: How I learned to start worrying about prompt formatting. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.11324.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2303.11366.

Zhi Rui Tam, Cheng-Kuang Wu, Yi-Lin Tsai, Chieh-Yen Lin, Hung-yi Lee, and Yun-Nung Chen. Let me speak freely? a study on the impact of format restrictions on performance of large language models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*, 2024. arXiv:2408.02442.

Hao Tang, Darren Key, and Kevin Ellis. WorldCoder, a model-based LLM agent: Building world models by writing code and interacting with the environment. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2402.12275.

Pedro A. Tsividis, Joao Loula, Jake Burga, Nathan Foss, Andres Campero, Thomas Pouncy, Samuel J. Gershman, and Joshua B. Tenenbaum. Human-level reinforcement learning through theory-based modeling, exploration, and planning. *arXiv preprint arXiv:2107.12544*, 2021.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.

Ruocheng Wang, Eric Zelikman, Gabriel Poesia, Yewen Pu, Nick Haber, and Noah D. Goodman. Hypothesis search: Inductive reasoning with language models. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2309.05660.

Archana Warrier, Dat Nguyen, Michelangelo Naim, Moksh Jain, Yichao Liang, Karen Schroeder, Cambridge Yang, Joshua B. Tenenbaum, Sebastian Vollmer, Kevin Ellis, and Zenna Tavares. Benchmarking world-model learning with environment-level queries. *arXiv preprint arXiv:2510.19788*, 2026. URL https://arxiv.org/abs/2510.19788.

Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, Ed H. Chi, Tatsunori Hashimoto, Oriol Vinyals, Percy Liang, Jeff Dean, and William Fedus. Emergent abilities of large language models. *Transactions on Machine Learning Research*, 2022. arXiv:2206.07682.

Yue Wu, Xuan Tang, Tom M. Mitchell, and Yuanzhi Li. SmartPlay: A benchmark for LLMs as intelligent agents. *arXiv preprint arXiv:2310.01557*, 2024.

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2405.15793.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2210.03629.

Sibo Zhu, Shicheng Fan, Xinyue Wang, Wenyi Wu, Kun Zhou, and Biwei Huang. RSIAgent: Autonomous exploration for recursive self-improvement in new environments. *arXiv preprint arXiv:2609.15364*, 2026. URL https://arxiv.org/abs/2609.15364.